

API Guide - Cover and Global Conventions

Source of truth: [docs/API.md](#)

Base URL (prod): <https://yassirhakimi-recruiteia-api.hf.space/api>

Global Conventions

- JSON for all endpoints except CV upload (multipart/form-data)
- Bearer token required for protected endpoints
- Do not set multipart Content-Type manually
- Poll async scoring sessions until completed/failed

Core Endpoint Groups

- 1) Authentication
- 2) Job Offers
- 3) CVs
- 4) Screening Sessions
- 5) Results + CSV Export
- 6) Health + Error Handling + CORS

Frontend Rule

Always map UI state to backend status codes and detail messages.
Never assume success without checking response shape.

Authentication Endpoints

POST /api/auth/register

Body: email, password, full_name, role

Success: user object

Errors: 400 duplicate email, 422 validation

POST /api/auth/login

Body: email, password

Success: access_token + user object

Error: 401 invalid credentials

Client behavior:

- persist token (localStorage)
- attach Authorization: Bearer <token> on protected requests

Auth guard: if 401 appears, clear token and redirect to login.

Job Offer Endpoints

- Offer CRUD
- POST /api/offers
 - GET /api/offers
 - GET /api/offers/{id}
 - PUT /api/offers/{id}
 - DELETE /api/offers/{id}

Important field name:
experience_required_years (not experience_years)

AI Extraction Endpoint
POST /api/offers/extract
Body: { text, lang? }
Returns normalized offer fields for direct form prefill.
Validation error: 400 when text is missing.

Integration tip: Use extract output to pre-populate offer creation UI.

CV Upload and Retrieval

POST /api/cvs (multipart/form-data)

Field: file (.pdf only, max 5MB)

Success includes candidate profile extraction fields.

Special states:

- is_duplicate true/false
- extraction_error (e.g. no_text_extracted)
- confidence_score (0.0-1.0)

GET /api/cvs

GET /api/cvs/{id}

Use these endpoints for candidate profile preview and audit.

Errors: 404 CV not found, 401 not authenticated.

Do not manually force multipart content-type in frontend code.

Sessions and Async Scoring Lifecycle

POST /api/sessions
Body: name, job_offer_id, cv_ids, weights
Constraint: weights must sum to 1.0 (+/- 0.01)
Errors: invalid IDs, invalid weights, missing offer

POST /api/sessions/{id}/score launches async scoring
Poll GET /api/sessions/{id} until status changes.
Status lifecycle:
pending → scoring → completed | failed
Conflict case: 409 scoring already in progress

Recommended polling interval: 3 seconds with cancel/timeout safeguards.

Results, Thresholds, and CSV Export

GET /api/sessions/{id}/results
If session not complete: returns status + empty results.
If completed: ranked candidates with score breakdown fields.
Threshold mapping:
- green >= 0.80
- orange 0.50-0.79
- red < 0.50

GET /api/sessions/{id}/export
Returns CSV download with rank, candidate fields,
score components, matched skills, and missing critical skills.

UI should display both percentage score and threshold color badge.

Errors, HTTP Codes, and Reliability

Universal Error Shape
{ "detail": "Error message" }

Core status codes:
200, 400, 401, 403, 404, 409, 413, 422, 500

Reliability Notes

- 401: clear token and redirect to login.
- 422: show field-level form errors.
- 409: show 'already in progress' and continue polling.
- 500: user-safe message + retry option.

CORS is open (*), proxy not required in development.

Error UX must be deterministic and actionable.

WordPress Integration Playbook

WordPress Frontend Checklist

- 1) Centralize base URL constant
- 2) Persist access_token and user metadata
- 3) Inject Authorization header for protected calls
- 4) Use FormData for CV upload (no manual multipart header)
- 5) Poll scoring sessions with progress indicator
- 6) Render threshold badges and score breakdown
- 7) Support CSV export flow
- 8) Implement auth guard + token-expiry redirect

End-to-End Sequence

Login → Offer extraction/creation → CV uploads → Session create
→ Score trigger → Poll status → Results → CSV export

This sequence should be implemented as a resilient state machine on UI.

Final gate: frontend behavior must exactly mirror API.md contracts.